Session 2 — Recap, Git step by step, branches

Git, Python Environments & Coding Best Practices

Guillaume Leveque

Session 2 of 4

Master of Data & AI — MSA-DATB07-03

Where am I? Paths (8 min)

A folder tree: your course folder

```
/c/Users/alice/              <- ~ (home)
`-- Documents/
    `-- courses/
        `-- git/
            |-- td1_answers.md
            `-- td1-shell-and-git/
                |-- README.md
                |-- project/
                `-- shell-lab/
                    |-- README.txt
                    `-- data/
                        `-- raw/
                            `-- sales_2024_q1.csv
```

- A **folder** (directory) contains files and other folders. The tree goes down from the top of the disk, `/`.
- A **path** is the address of a file: the folders to go through, separated by `/`.
- The terminal is always *in* one folder: the **current folder**. `pwd` prints it.

On macOS the home folder is `/Users/alice`. On Windows, Git Bash writes `C:\Users\alice` as `/c/Users/alice`.

Absolute or relative

Absolute: starts with / (or ~), from the top. Works from anywhere.

```
/c/Users/alice/Documents/courses/git
~/Documents/courses/git
```

Relative: starts from the current folder. Its meaning depends on where you are.

```
shell-lab/README.txt
../td1_answers.md
```

`/`	top of the disk; separator
`~`	your home folder
`.`	the current folder
`..`	the folder above
`../..`	two folders above

Tip — never type a long path

Type cd and a space, then **drag the folder** from the file explorer (Finder on macOS) onto the terminal and press `Enter`: the absolute path is typed for you. Or type the first letters and press `Tab`.

Check yourself: where am I? (hands up)

Check yourself — same tree as two slides ago; you are in `~/Documents/courses/git/td1-shell-and-git`

1. You type `cd shell-lab/data`. What does `pwd` print?
2. From there, which command reads `td1_answers.md` without moving: `cat td1_answers.md`, `cat ../td1_answers.md` or `cat ../../../td1_answers.md`?
3. You type `cd ~/Documents/courses/git`. Does it work from any folder? Why?

Check yourself: the answers

1. `/c/Users/alice/Documents/courses/git/td1-shell-and-git/shell-lab/data`: the relative path `shell-lab/data` is added to the current folder.
2. `cat ../../../td1_answers.md`: three levels up, `data` → `shell-lab` → `td1-shell-and-git` → `git`.
3. Yes: it starts with `~`, the home folder, so it does not depend on where you are.

The shell commands to know (5 min)

The terminal inside VS Code

Open it

- menu *Terminal* → *New Terminal* (works everywhere)
- or `Ctrl+ù` on a French keyboard, `Ctrl+`` on an English one

Check it is bash

- `echo $0` prints `bash` (Windows) or `zsh` (macOS)
- an empty line: it is PowerShell (next point)

Windows: make Git Bash the default

1. `Ctrl+Shift+P`, type *default profile*
2. choose *Terminal: Select Default Profile*
3. choose *Git Bash*
4. close the terminal (trash icon), open a new one

Ten commands

`pwd`	where am I?
`ls`, `ls -la`	what is here? (`-a`: also the hidden files, `-l`: details)
`cd folder`, `cd ..`, `cd ~`	go into a folder, go up one level, go home
`mkdir name`	create a folder
`cat file`	print a short file
`code file`, `code .`	open a file, or the current folder, in VS Code
`echo "text" > file`	write a text into a file (`>>`: add at the end)
`Tab`, `Up`	complete a name, recall the previous command
`q`	leave a long list (`(END)`, `:`)
`Ctrl+C`	stop a running command

`code: command not found`? Open the file from VS Code instead (*Explorer* panel on the left), and read it with `cat`. All the other commands are on the cheat-sheet of session 1.

Why Git, in one picture (7 min)

The problem, and what Git does

Without Git:

```
analysis.py
analysis_v2.py
analysis_v2_bob.py
analysis_final.py
analysis_final_REAL.py
```

Which one is current? What changed, who, why? How to get Monday's version back?

With Git: **one** file `analysis.py`, and a history of saved versions, each one with

- an identifier (`40a9ded`, its *hash*),
- an author and a date,
- a message: what changed and why.

A saved version is a **commit**: a save point of the whole project, as in a video game.

The three areas



- You edit files freely: nothing is saved in Git until you ask.
- `git add`: "this change goes into my next save".
- `git commit`: save everything that was added, as one commit, with a message.
- The history lives in a hidden folder `.git`, inside the project folder.

The loop, step by step (22 min)

Step 1 — Create a project and a repository

```
mkdir demo_week      # a new folder for the project
cd demo_week         # go into it
git init             # turn this folder into a Git repository
```

```
Initialized empty Git repository in C:/Users/alice/demo/demo_week/.git/
```

- `git init` creates the hidden folder `.git`: the history will live there. `ls -la` shows it.
- Run it **once per project**, inside the project folder. Check with `pwd` first: never in your home folder.

Step 2 — A first file: untracked

```
echo "Monday: plan" > notes.txt      # create a file with one line
git status                           # what does Git see?
```

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        notes.txt
```

Untracked: the file exists in the folder, but Git does not follow it yet. `git status` is the command to run whenever you wonder "where am I?" in Git.

Step 3 — git add: into the staging area

```
git add notes.txt
git status
```

```
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   notes.txt
```

Changes to be committed = the staging area: what the next commit will contain. Nothing is saved in the history yet.

Step 4 — git commit: the first save point

```
git commit -m "Add Monday"      # -m = the message, in quotes
git log --oneline               # the history: one line per commit
```

```
[main (root-commit) 379ea73] Add Monday
 1 file changed, 1 insertion(+)
 create mode 100644 notes.txt

379ea73 (HEAD -> main) Add Monday
```

- 379ea73: the identifier of the commit (yours is different).
- `main`: the name of the line of work. `HEAD -> main`: you are on it.
- A good message starts with a verb and says what the commit does.



Step 5 — Edit, then look at the change

```
echo "Tuesday: code" >> notes.txt    # >> adds a line at the end
git status                           # modified, not staged
git diff                             # what exactly changed?
```

```
Changes not staged for commit:
        modified:   notes.txt
```

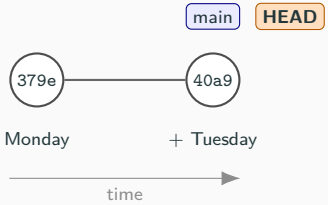
```
@@ -1 +1,2 @@
 Monday: plan
+Tuesday: code
```

- **modified**: Git knows the file, and it changed since the last commit.
- `git diff`: lines starting with + were added, with − removed. Press `q` if the list is long.

Step 6 — Second commit: a history

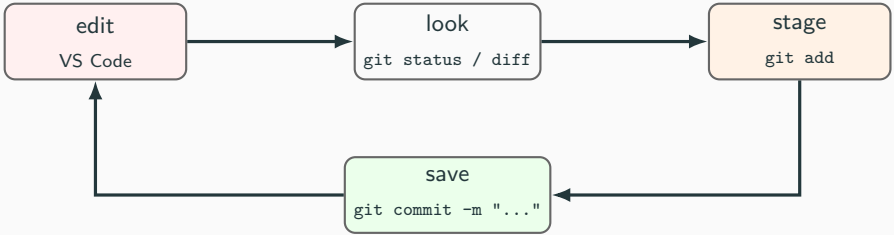
```
git add notes.txt
git commit -m "Add Tuesday"
git log --oneline
```

```
40a9ded (HEAD -> main) Add Tuesday
379ea73 Add Monday
```



Newest on top in `git log`; left to right on the drawings. The label `main` moved forward to the new commit.

The loop, to remember



- One change = one commit: “Add Tuesday”, not “stuff”.
- `git status` before and after every step: it tells you where you are, and what to type next.
- `git log --oneline` after: your history.

.gitignore: what never goes into Git

```
# .gitignore: a text file at the root of the project, one pattern per line
__pycache__/
outputs/
*.png
.env
```

- Files that are **generated** (caches, outputs), **big** (raw data) or **secret** (passwords in `.env`).
- Ignored files disappear from `git status`: `git add .` will not take them.
- Commit the `.gitignore` itself, so the whole team ignores the same files.

Undo (8 min)

Undo an edit not committed: git restore

```
echo "oops" > notes.txt      # mistake: > erased the file and wrote "oops"
cat notes.txt                # oops
git restore notes.txt        # back to the version of the last commit
cat notes.txt
```

```
Monday: plan
Tuesday: code
```

Pitfall — gone for good

`git restore` throws away the edits made since the last commit. Fine here; but Git cannot bring back an edit that was never committed. Commit often.

Undo a bad commit: git revert

```
echo "Friday: party all day" >> notes.txt
git add notes.txt
git commit -m "Add Friday"      # a commit we regret
git revert HEAD                 # HEAD = the last commit; VS Code opens: save, close
git log --oneline
```

```
d9fc1d4 (HEAD -> main) Revert "Add Friday"
27359b2 Add Friday
40a9ded Add Tuesday
379ea73 Add Monday
```

- `revert` adds a **new** commit that does the opposite. Nothing is erased: the history only grows.
- For an older commit, give its identifier: `git revert 27359b2`.
- The default way to undo a commit, always safe.

Safe or not: the summary, before branches

command	effect	risk
`git status`, `git diff`, `git log`	read only	**safe**
`git add`, `git commit`	save a new version	**safe**
`git revert <id>`	new commit cancelling one commit	**safe** (the default way to undo)
`git restore file`	throw away the edits not committed	**destroys work** (those edits only)
`rm -rf .git`	delete the whole history	**destroys work**

Keep this table: it grows with branches today, with GitHub in session 3.

Branches (12 min)

The problem branches solve

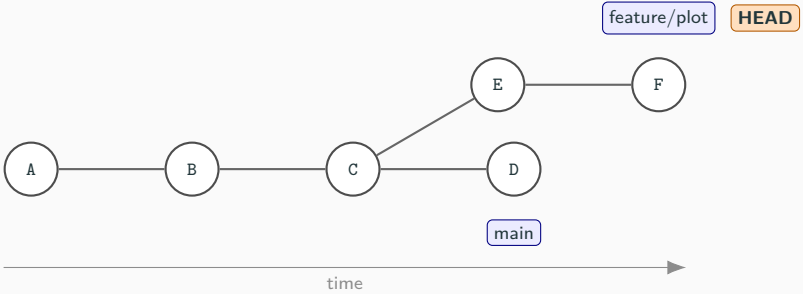
Alice works on a chart, Bob on a bug fix, in the same project.

- If both commit on the same line of work, half-finished work gets mixed with finished work.
- `main` should always contain a version that works.

A **branch** is a separate line of work: you commit there freely, and you bring the work back into `main` when it is ready.

- **Isolate**: `main` stays clean while you try things.
- **Work in parallel**: three teammates, three branches.
- **Throw away** cheaply: the branch is dropped, `main` never saw it.

A branch is a label on a commit



- After C the history splits in two. E and F exist only on `feature/plot`.
- A **branch** (`main`, `feature/plot`) is a *label* stuck on its last commit. Creating one costs nothing.
- **HEAD** = "you are here". When you commit, only the label of the branch you are on moves forward.

Three commands, and a way to see everything

```
git switch -c feature/wednesday   # -c = create a new branch here AND go to it
git switch main                   # go to an existing branch
git branch                        # list the branches; * = the one you are on
git branch -a                     # -a = all: also the branches on GitHub (session 3)
```

```
git config --global alias.lg "log --oneline --graph --all"   # once per computer
git lg                            # = git log --oneline --graph --all
```

`--oneline`	one line per commit
`--graph`	draw the lines between the commits, on the left
`--all`	all the branches, not only the one you are on

Switching changes the files in your folder to the version of that branch. Branch names: lowercase, with `/` and `-`, e.g. `fix/empty-csv`.

`git branch feature/x` also creates a branch, but you *stay* on the current one: your next commit goes there, not on `feature/x`. `git switch -c` creates *and* goes there: one step, no surprise.

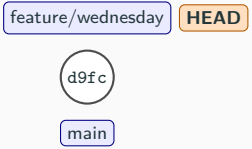
Step 7 — A branch, then a commit on it

```
git switch -c feature/wednesday      # (a)
echo "Wednesday: test" >> notes.txt
git add notes.txt
git commit -m "Add Wednesday"        # (b)
git lg
```

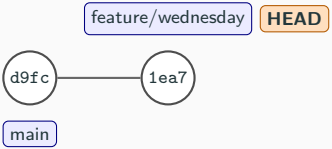
```
* 1ea70ef (HEAD -> feature/wednesday) Add Wednesday
* d9fc1d4 (main) Revert "Add Friday"
* 27359b2 Add Friday
...
```

On `main`, `cat notes.txt` shows no Wednesday: `main` is one commit behind.

(a) after `git switch -c`: a new label on the *same* commit, and `HEAD` goes to it.



(b) after `git commit`: only the label you are on moves forward.



Check yourself (hands up)

Check yourself — three questions, thirty seconds each

1. You are on `feature` and run `git commit`. Which label moves: `feature` or `main`?
2. You run `git switch main`. What happens to the files in your folder? Are the commits of `feature` lost?
3. In `git lg`, how do you see which branch you are on?

Check yourself: the answers

1. `feature`, the branch you are on. `main` does not move.
2. The files change to `main`'s version. The commits of `feature` are not lost: the `feature` label still points to them, `git switch feature` brings them back.
3. The label `HEAD -> name` is on the commit of the branch you are on.

Merging (10 min)

Merging: bring a branch into another

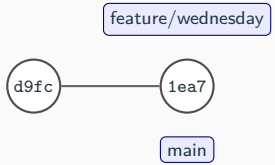
```
git switch main                 # 1. go to the branch that RECEIVES the work
git merge feature/wednesday     # 2. bring the other branch INTO it
```

- `main` changes, `feature/wednesday` does not. You merge *into* the branch you are on.
- Two possible results, depending on the shape of the history: **fast-forward** or **three-way merge**.

Step 8 — Case 1: fast-forward

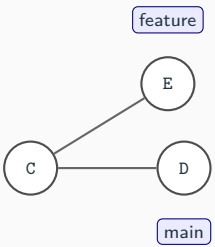
```
git switch main
git merge feature/wednesday
```

```
Updating d9fc1d4..1ea70ef
Fast-forward
 notes.txt | 1 +
 1 file changed, 1 insertion(+)
```

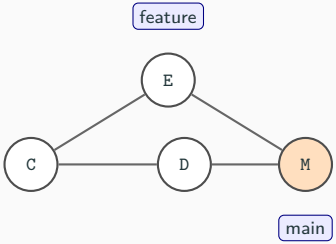


Nothing happened on `main` since the branch was created: Git simply moves the `main` label forward. **No new commit**; the history stays a straight line.

Case 2: three-way merge



before: both moved since C



after: a *merge commit* M joins the two lines

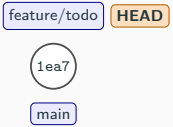
- Both branches got new commits since C. Git combines the changes of D and E into a new commit M, the **merge commit**.
- It works by itself when the two sides changed different files, or different places of a file. Otherwise: a conflict.

Step 9 — Both branches move

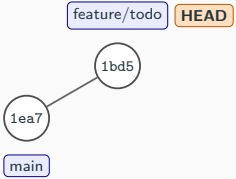
```
git switch -c feature/todo      # (1)
echo "buy milk" > todo.txt
git add todo.txt
git commit -m "Add todo list"   # (2)
git switch main                 # (3)
echo "Thursday: review" >> notes.txt
git add notes.txt
git commit -m "Add Thursday"    # (4)
```

Watch `HEAD`: it is the label that moves when you commit.

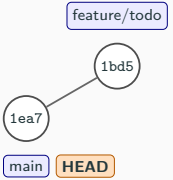
(1) new branch, same commit



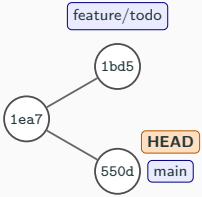
(2) commit on `feature/todo`



(3) back on `main`: only `HEAD` moves



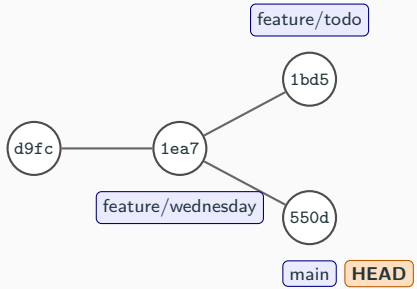
(4) commit on `main`: the history splits



Reading git lg: the same drawing, turned

```
git lg
```

```
* 1bd5616 (feature/todo) Add todo list
| * 550da02 (HEAD -> main) Add Thursday
|/
* 1ea70ef (feature/wednesday) Add Wednesday
* d9fc1d4 Revert "Add Friday"
```



in `git lg`	on the drawing
each `*`	a commit (a circle)
`|`, `/` and `\`	the lines between commits
`(feature/todo)`, `(HEAD -> main)`	the labels: branches, and where you are
newest on **top**, time goes up	newest on the **right**, time goes right

Step 10 — The merge commit

```
git merge feature/todo     # VS Code opens with a ready message: save, close the tab
git lg
```

```
Merge made by the 'ort' strategy.
 todo.txt | 1 +

*   aeb69d3 (HEAD -> main) Merge branch 'feature/todo'
|\
| * 1bd5616 (feature/todo) Add todo list
* | 550da02 Add Thursday
|/
* 1ea70ef (feature/wednesday) Add Wednesday
```

- aeb69d3 is the merge commit: the two lines join there. `main` now has `todo.txt` *and* Thursday.
- The terminal waits while VS Code is open: save and close the tab to get the prompt back.

Conflicts (13 min)

What a conflict is

A conflict happens when the two branches changed **the same lines**, or lines right next to each other, in two different ways. Git cannot know which version is right: it asks you.

Git compares *lines*, not meaning: running the code after a merge is still your job.

Not an error

A conflict is a question: “which version do you want?”. You answer by editing the file. Nothing is broken, and you can always cancel.

Step 11 — Provoke a conflict

```
git switch -c feature/rename
code notes.txt          # line 1: "Monday: plan" -> "Monday: planning", save
git add notes.txt
git commit -m "Rename Monday task"
git switch main
code notes.txt          # line 1: "Monday: plan" -> "Monday: meeting", save
git add notes.txt
git commit -m "Monday is a meeting"
git merge feature/rename
```

```
Auto-merging notes.txt
CONFLICT (content): Merge conflict in notes.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Both branches changed the first line of `notes.txt`, in two different ways.

Step 12 — Read the conflict

```
git status
```

```
You have unmerged paths.
Unmerged paths:
        both modified:   notes.txt
```

```
cat notes.txt
```

```
<<<<<<< HEAD
Monday: meeting
=======
Monday: planning
>>>>>>> feature/rename
Tuesday: code
Wednesday: test
Thursday: review
```

`<<<<<<< HEAD` ... `=======`	the version of the branch you are on (`main`)
`=======` ... `>>>>>>> feature/rename`	the version coming from the other branch
the other lines	merged without problem

Step 13 — Resolve, add, commit

1. Open the file. Keep one side, the other, or write a mix. **Delete the three marker lines.**
2. `git add notes.txt`: "this file is resolved".
3. `git commit`: the merge commit. VS Code opens with a ready message: save, close.

```
code notes.txt      # line 1 becomes "Monday: meeting, then planning"; no marker left; save
git add notes.txt
git commit
git lg
```

```
*   1306809 (HEAD -> main) Merge branch 'feature/rename'
|\
| * 5b03bbe (feature/rename) Rename Monday task
* | c74c55a Monday is a meeting
|/
*   aeb69d3 Merge branch 'feature/todo'
```

Resolving in VS Code

The file opens with coloured blocks and clickable actions above each one:

- *Accept Current Change*: keep your side (`HEAD`)
- *Accept Incoming Change*: keep the other branch's side
- *Accept Both Changes*: one after the other

The buttons remove the markers for you. Often the right answer is a mix that you write yourself.

Pitfall — clicking is not finishing

Whatever you click, the rule is the same: no marker left in the file, then `git add`, then `git commit`. Check with `git status`.

Cancel, and the classic mistake

```
git merge --abort           # cancel the merge: back exactly to the state before "git merge"
grep -n "<<<<<<<" notes.txt   # any marker left? (-n = with the line number)
```

- Lost in a conflict? `git merge --abort`, breathe, and start the merge again.

Pitfall — committing the markers

If you `git add` a file that still contains `<<<<<<<`, Git commits it without complaining. Python then fails with a `SyntaxError` on that line. Search for markers before committing, and run your code.

Wrap-up (5 min)

Safe or not: the summary, with branches

command	effect	risk
`git status`, `git diff`, `git log`, `git lg`	read only	**safe**
`git branch`, `git branch -a`	list the branches (read only)	**safe**
`git add`, `git commit`	save a new version	**safe**
`git revert <id>`	new commit cancelling one commit	**safe** (the default way to undo)
`git switch`, `git switch -c`	go to a branch, create one	**safe**
`git merge`	bring a branch in (maybe a merge commit)	**safe**
`git merge --abort`	cancel a merge in progress	**safe**
`git restore file`	throw away the edits not committed	**destroys work** (those edits only)
`git add` + `commit` with `<<<<<<<` left	the commit contains a file that does not run	**breaks the code** (fix: a new commit)

Read this repository

```
$ git log --oneline --graph
*   9c1d2e7 (HEAD -> main) Merge branch 'feature/stats'
|\
| * e6adc17 (feature/stats) Add region filter to total_revenue
* | d37005f Round total_revenue to the cent
|/
* a97a373 Add command-line entry point
$ python analysis.py
  File "analysis.py", line 13
    <<<<<<< HEAD
SyntaxError: invalid syntax
```

Check yourself — the kind of question of the quiz

1. Was this a fast-forward or a three-way merge? How can you tell from the graph?
2. What went wrong during the merge? Which step was skipped?
3. How do you repair it without rewriting history?

Read this repository: the answers

1. A **three-way merge**: the graph splits after `a97a373` and joins again in `9c1d2e7`, a merge commit with two parents. A fast-forward would leave a straight line.
2. The file was committed with the conflict markers still in it: the step "delete the three marker lines" was skipped before `git add` and `git commit`.
3. Open `analysis.py`, keep the right lines, delete the markers, check that it runs, then `git add analysis.py` and `git commit`: a new commit repairs it. Before committing, `grep -n "<<<<<<<" analysis.py` would have shown the markers.

Take-aways and TD2

Remember

- `pwd` first; absolute = from `/` or `~`, relative = from here
- edit → `status` → `add` → `commit` → `log`
- undo: `restore` (not committed), `revert` (committed)
- branch = a label; `HEAD` = you are here
- merge *into* the branch you are on
- conflict: edit, remove the markers, `add`, `commit`; or `--abort`

Next session: working as a team on GitHub.

TD2 (1 h 30)

1. check your setup (15 min)
2. Git step by step, your own project (25 min)
3. branches and two merges (25 min)
4. the conflict (20 min)
5. before session 3: a GitHub account (5 min)

Bonus if you finish early. Not graded.

Appendix: for the curious

Appendix: for the curious

Not needed for the quiz

The next slides are extras: they are not presented in the lecture, and the quiz does not ask about them. Everything you need is in the slides before this one.

Read them if you are curious, or if you reach the bonus of TD2 (if you get that far: well done).

- looking closer: `diff --staged`, `show`, `HEAD~1`, `branch -d`
- `git reset`, the exception to `git revert`
- `git stash`, `git rebase`
- fewer conflicts, detached HEAD, comparing branches, merge options, conflict tools, cleaning commits

Appendix — `diff --staged`, `show`, `HEAD~1`

```
git diff --staged           # what IS staged: exactly what the next commit will contain
git restore --staged f.py   # take f.py out of the staging area; the edit stays in the file
git show 40a9ded            # one commit in detail: who, when, message, and the changes
git show HEAD~1             # HEAD~1 = the commit before the current one
git branch -d feature/todo # delete the label of a merged branch (its commits stay)
```

`git diff` alone shows the edits *not* staged yet; `--staged` shows the staged ones. Good habit: read `git diff --staged` before each commit.

Appendix — git reset: the exception

```
git reset --soft HEAD~1    # undo my last commit, keep its changes staged, to redo it
    properly
```

- `reset` moves the branch label backwards: the commits after that point disappear from the history.
- Acceptable only for a commit you *just* made, that exists only on your laptop (forgot a file, wrong message). `--soft` keeps all your work.
- `git reset --hard` also throws away every edit not committed: avoid it. The default way to undo stays `git revert`.

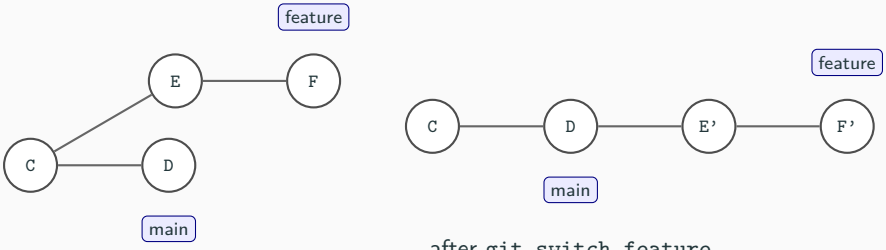
Appendix — git stash: put your edits aside

```
$ git switch feature/todo
error: Your local changes to the following files would be overwritten by checkout:
        notes.txt
Please commit your changes or stash them before you switch branches.
```

```
git stash          # put the edits not committed aside; the folder is clean again
git switch feature/todo     # do the urgent thing, then come back
git switch main
git stash pop      # bring the edits back
```

Git refuses to switch because your edit would be overwritten. A stash does not appear in `git log`: bring it back the same day.

Appendix — Rebase: replay my commits on top of main



Git replays the commits of your branch (E, F) after the last commit of `main`. The copies E', F' have **new identifiers**. Result: a straight history.

The golden rule

Rebase only commits that are still only on your laptop. Once a commit is on GitHub (session 3), a teammate may have downloaded it: rebase would replace it with a copy, and their history would no longer match yours.

Appendix — Fewer conflicts, smaller conflicts

```
git switch feature/plot
git merge main              # bring the new commits of main INTO my branch
```

- **Bring `main` into your branch regularly**: conflicts stay small, and you solve them on your side, where you know the code.
- **Small branches, merged often.** A branch that lives two days conflicts less than one that lives two weeks.
- **Do not reformat a whole file** in a feature branch: every line changes, every other branch conflicts.
- **Talk.** Two people changing the same function should know it before, not after.

Appendix — Detached HEAD

```
git switch --detach v0.1     # look at an old commit (or: git checkout a97a373)
# You are in 'detached HEAD' state...
```

- `HEAD` points to a commit instead of a branch. Fine to look around and run old code.
- Commits made here belong to no branch: easy to lose.
- Way out: `git switch main` to leave, or `git switch -c rescue` to keep what you committed there.

The message looks scary but is not an error: it tells you exactly these two commands.

Appendix — Comparing branches

```
git log --oneline main..feature/stats   # commits feature/stats has and main does not
git diff --stat main...feature/plot     # files changed on feature/plot since it split from
    main
git merge-base main feature/plot        # the common starting point of the two branches
git branch --merged                     # branches already merged into the current one
```

- Two dots `A..B`: the commits of B that A does not have.
- Three dots in `git diff`: what changed on one side since the common starting point.

Appendix — Merge options

```
git merge --no-ff feature/readme   # always create a merge commit, even when a fast-forward
    is possible
git log --oneline --merges         # only the merge commits
git log --oneline --first-parent   # main's own line, each merged branch shown as one commit
```

Teams often use `--no-ff` (GitHub's merge button does it): each feature stays visible as one bump in the graph.

Appendix — More conflict tools

```
git diff                              # during a conflict: both sides of each block
git log --merge --oneline             # the commits of both sides touching the conflicted
    files
git checkout --ours   analysis.py     # keep our whole file (then git add)
git checkout --theirs analysis.py     # keep their whole file (then git add)
```

`--ours` / `--theirs` throw away one side of the file entirely: use them only when you are sure.

Appendix — Cleaning your commits: rebase -i, amend

```
git commit --amend -m "Better message"   # fix the message of the last commit (or add a
    forgotten file)
git rebase -i HEAD~3                     # rework the last three commits in an editor
```

```
pick 1a2b3c4 Add average_order_value
squash 5d6e7f8 fix typo
reword 9a8b7c6 Add tests
```

`pick` keep, `squash` melt into the previous one, `reword` change the message, `drop` remove. Both rewrite history: only on commits nobody else has.